

Project 3 – IFC and Noninterference

Benjamin Deutsch (12215881)

TU Wien

4 Noninterference Analysis

5 Relaxing Noninterference

5.1 Tax Bracket Calculation

Nia analysis of program in Figure 4 will output that the program is interferant. As in the lecture, we can represent the states of the memory for each variable using $\oplus$, $\odot$ and $\ominus$. Consider the following two states of the memory:

$$\begin{aligned} m_1 &: \{\{ \textcolor{blue}{lwinner} : \oplus \odot \ominus, \textcolor{red}{hbid A} : \ominus, \textcolor{red}{hbid B} : \oplus \}\} \\ m_2 &: \{\{ \textcolor{blue}{lwinner} : \oplus \odot \ominus, \textcolor{red}{hbid A} : \oplus, \textcolor{red}{hbid B} : \ominus \}\}. \end{aligned}$$

To process line 1, we apply Propagate-Taint on both traces to get the following states:

$$\begin{aligned} m_1 &: \{\{ \textcolor{blue}{lwinner} : \ominus, \textcolor{red}{hbid A} : \ominus, \textcolor{red}{hbid B} : \oplus \}\} \\ m_2 &: \{\{ \textcolor{blue}{lwinner} : \ominus, \textcolor{red}{hbid A} : \oplus, \textcolor{red}{hbid B} : \ominus \}\}. \end{aligned}$$

To process line 3, we need to apply the Raise-Context-Step rule, since the control flow depends on tainted variables. Therefore, until joining at line 11, the context is now tainted. Note that the trace starting with m_1 , is now before line 4, since the condition is true in that trace. The other trace is before line 6, since the condition is false in that trace.

Now, we can apply Propagate-Taint on line 4 for the first trace to get

$$m_1 : \{\{ \textcolor{red}{lwinner} : \oplus, \textcolor{red}{hbid A} : \ominus, \textcolor{red}{hbid B} : \oplus \}\}.$$

lwinner is now tainted, since it was assigned under a tainted context. At this point, the first trace is before line 11, waiting to join with the second trace.

On the second trace, we can apply Propagate-Taint to process line 6, since the current context is already tainted. As the condition is true, the trace is now before line 7. To process line 7, we can apply Propagate-Taint again to get

$$m_2 : \{\{ \textcolor{red}{lwinner} : \odot, \textcolor{red}{hbid A} : \oplus, \textcolor{red}{hbid B} : \ominus \}\}.$$

Now, both traces are before line 11. Therefore, we can apply the Join rule, to get

$$\begin{aligned} m_1 &: \{\{ \textcolor{red}{lwinner} : \oplus, \textcolor{red}{hbid A} : \ominus, \textcolor{red}{hbid B} : \oplus \}\} \\ m_2 &: \{\{ \textcolor{red}{lwinner} : \odot, \textcolor{red}{hbid A} : \oplus, \textcolor{red}{hbid B} : \ominus \}\}. \end{aligned}$$

Note that, since the value of *lwinner* differs in the two traces and it is tainted in both traces, it remains tainted after the join. After processing the two remaining assignments using Propagate-Taint, we get the following states:

$$\begin{aligned} m_1 &: \{\{ \textcolor{red}{lwinner} : \oplus, \textcolor{blue}{hbid A} : \odot, \textcolor{blue}{hbid B} : \odot \}\} \\ m_2 &: \{\{ \textcolor{red}{lwinner} : \odot, \textcolor{blue}{hbid A} : \odot, \textcolor{blue}{hbid B} : \odot \}\}. \end{aligned}$$

Since the public variable *lwinner* is tainted, the program is interferant.

5.2 Controlled Information Leaks: Abstract Noninterference

My implementation first checks whether the variables *hbid A* and *hbid B* are present in the variables list. If so, then *hbid A* and *hbid B* are added to the observed variable list, since those are needed to compute ϕ . Also, if *hbid A* and *hbid B* are present, `auction_winner` will return a z3 expression calculating the winner of the auction. If *hbid A* and *hbid B* are not present, the the function returns 0. Finally, `relate_executions` checks for the presence of *hbid A* and *hbid B*. If they are not present, it simply checks low variable equivalence. If *hbid A* and *hbid B* are present, then it additionally checks that the winner of the auction is the same in both traces.

simple_winner.while. This program is wrongly labeled as interferant, since the if statement is the last statement of the program. The abstraction of `Seq( $c_1$ ,  $c_2$ )` only checks whether the c_1 might require a join, but not whether c_2 requires one. Therefore, if the statement requiring a join is the last statement of the program, the Join rule is never applied, which is why *lwinner* stays tainted in the end.

To fix this, we could

- change the `Seq` abstraction to also check whether c_2 is a possible join point, or
- add a trivial statement (e.g. a self-assignment, like it is done in `simple.while`) after the if statement, enabling the Join rule to be applied.